Min-Conflicts Local Search on a Terrain Map-Coloring CSP

Jacob A. Geldbach
College of Engineering: CS
University of Idaho
Moscow Idaho, United States
geld8788@vandals.uidaho.edu

***Abstract*—Min-Conflicts is a local search algorithm useful for solving constraint satisfaction problems (CSPs). In this project, Min-Conflicts was applied to a terrain detailed grid map coloring CSP, where different terrains acted as the variables and the constraints were related to which terrains could be adjacent to each other. The algorithm was tested on 5 differnt map sizes, with and without diagonal neighbors.**

***Index Terms*—Local Search, Contraint Satisfaction Problem, Min-Conflicts, Map Coloring, Convergence**

I. Introduction

In this project, the Min-Conflicts algorithm was successfully implemented in order to solve a terrain map-coloring CSP problem. The terrains were represented as a grid of cells which is generated randomly and represented in real time as the algorithm runs. The python process uses the Pygame visualization library to display the initial random terrain assignment for the inputted map size, then continues to display an update of the grid every 50 steps until either the algorithm finds a solution or the stopping criterion is met. The constraints for this problem are the adjacency rules for the terrains defined below in Table I. This projects python process has the ability to run headless without generating the visualization at all, this was useful for the data gathering portion of the project where I ran 16 trials for each map size.The ability to run the algorithm headless allowed for the trials to be ran in parallel and gather the data much faster as on the larger map sizes the Min-Conflicts algorith took a long time to either converge or reach the stopping criterion, the reason for this discussed in the below sections. The algorithm was successful in converging on a solution in all 10 conditions tested, and the data gathered allowed for a clear representation of how this CSPs time complexity scales with the size of the map or the number of variables constrained.

TABLE I
Terrain Adjacency Compatibility Matrix

Terrain	Water	Beach	Lowland	Forest	Hills	Mountains
Water	✓	✓	×	×	×	×
Beach	✓	✓	✓	×	×	×
Lowland	×	✓	✓	✓	×	×
Forest	×	×	✓	✓	✓	×
Hills	×	×	×	✓	✓	✓
Mountains	×	×	×	×	✓	✓

A ✓ marks a legal adjacency (elevation differs by at most one step); × marks a constraint violation. Every terrain is trivially compatible with itself.

II. Algorithm Description

Min-Conflicts being a local search algorithm starts in a problem search space by randomly choosing a potential solution, in this case that is a complete assignment of values to the variables. The algorithm must then "grade" or determine how many constraint violations this grid state has, this is where the majority of the time complexity comes from. The algorithm must first iterate over each cell in the grid and check each of it's neighbors to see if any of the contraints are violated, each cell with a constraint violation is then added to a list of conflicted cells. A conflicted cell is then chosen at random and the algorithm iterates over all the possible values for that cell and determines which value would improve the overall conflicted state of the current grid state. These two steps result in time complexities described by $O(n * d)$ and $O(k * d)$ respectively, where n is the number of cells or variables, d is the number of neighbors for each cell, and k is the number of possible values for each variable or terrains each cell can be colored. Due to in this problem k being a constant value of 5, and d being a constant value of 4 or 8, the time complexity for the step of finding a new terrain for the chosen conflicted cell is negligible and considered constant $O(1)$. Min-Conflicts being a local search algorithm benefits from the memory efficiency of only storing the current state of the grid and that of the candidate state being tested, therefore the space complexity is $O(n)$ where n is the number of cells on the grid. Min-Conflicts is not guaranteed to find a solution if one exists, this is due to there before possible plateaus or local maximum in the search space where the algorithm

can get stuck and not be able to find a better solution. Due to this fact a stopping criterion is essential to prevent the algorithm from running indefinitely. In this project I chose a stopping criterion of steps since improvement. Essentially if a certain number of iterations or steps have passed without the algorithm finding a better solution, i.e. decreasing the total number of conflicts, then the algorithm will stop and report a failure to converge. Due to the number of variable scaling of Min-Conflicts time complexity, I also have the stopping criterion of steps since improvement scaling with the size of the grid, where $steps_since_improvement = w * h$ where w and h are the width and height of the grid. I chose this so that the algorithm has a fair chance to converge on larger variable conditions and so that algorithm could stop when in a clear plateau or local maximum instead of continuing on for 10s of thousands of more iterations until a static maximum number of steps criteria was hit. A key benefit of Min-Conflicts shows itself in this problem space especially in the larger map size conditions, and it lies in the first step of starting from a random state of values assigned to the variables. By starting in this random potential solution, the exponential time complexity of building a solution to the CSP incrementally (really finding the solution) by traversing a tree of potential value assignments is able to be avoided. Restated, although this approach is complete, to incrementally build a grid of terrains that do not have any conflicts would be of time and space complexity of $O(k^n)$ where n is the total number of cells in the grid and k is the 6 terrain choices. Take the 200x200 largest grid condition, where $n = 40,000$, to incrementally find the solution the time and space complexities are $O(6^40,0000)$, practically INF in terms of even modern day computers. In other words, Min-Conflicts sacrificing completeness, allows for that condition in practice to become solvable. In this project, a greedy flag or non-classic Min-Conflicts argument was added which would instruct the algorithm to pick the next conflicted cell from a list of the *most* conflicted cells (i.e. cells were sorted by the number of neighbors they were conflicting with) instead of picking *any* conflicted cell. The thought process behind this addition was that by tackling the most conflicted cells first it may speed up time or lessen the number of steps before convergence. Counter-intuitively this caused the algorithm to plateau more often, which will be discussed in more depth in later sections.

III. Results

A. Map Visualizations

B. Results Table

C. Convergence Graph

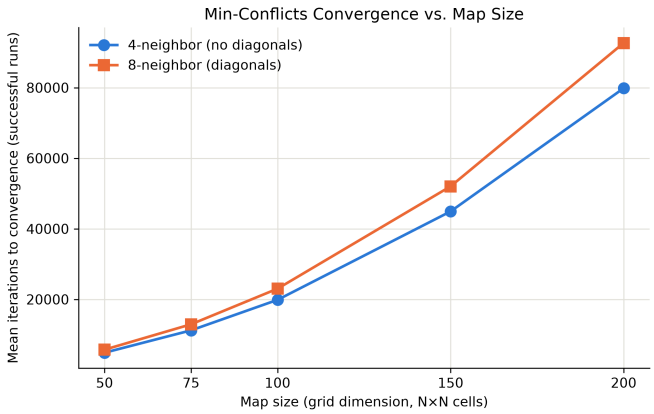

Fig. 2.

IV. Discussion

Use of AI

Coding

What Worked and Didn't Work

Writing

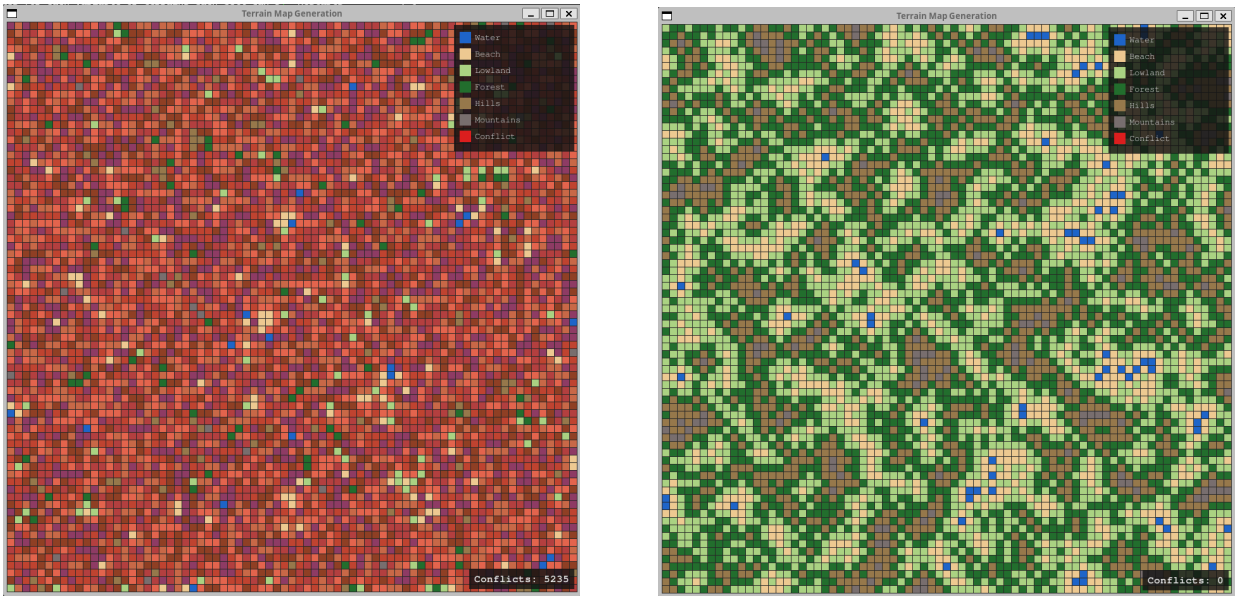

Fig. 1. Left: Beginning random terrain assignments for the the 75x75 grid size Right: Successful convergence of a solution

TABLE II
MIN-CONFLICTS — CONVERGENCE RESULTS (16 TRIALS PER CONDITION)

Map Size	**Diagonals**	**Avg. Iterations (success)**	**Failures (out of 16 trials)**	**Stopping Criterion**
50×50	Off	4892	2	Solved / steps-since-improvement $\geq w \times h$
50×50	On	5752	0	Solved / steps-since-improvement $\geq w \times h$
75×75	Off	11225	3	Solved / steps-since-improvement $\geq w \times h$
75×75	On	12939	0	Solved / steps-since-improvement $\geq w \times h$
100×100	Off	19877	3	Solved / steps-since-improvement $\geq w \times h$
100×100	On	23038	0	Solved / steps-since-improvement $\geq w \times h$
150×150	Off	44937	4	Solved / steps-since-improvement $\geq w \times h$
150×150	On	52045	0	Solved / steps-since-improvement $\geq w \times h$
200×200	Off	79852	12	Solved / steps-since-improvement $\geq w \times h$
200×200	On	92672	0	Solved / steps-since-improvement $\geq w \times h$